Corso di Fondamenti di Informatica 2
CdL Ingegneria Informatica
Ing. Franco Zambonelli

ARCHITETTURA DEGLI ELABORATORI: ASSEMBLER PER 8086/8088

Lucidi Realizzati in Collaborazione con:

Prof. Letizia Leonardi
Università di Modena

Prof. Antonio Corradi
Università di Bologna

LINGUAGGIO ASSEMBLER PER 8086/8088

Linguaggio Macchina

- insieme di istruzioni che un elaboratore è in grado di eseguire direttamente
- strettamente correlato alla realizzazione fisica dell'elaboratore

Per scrivere un codice in linguaggio macchina è necessario:

- conoscere l'architettura della macchina
- ragionare in termini del comportamento dell'elaboratore
- conoscere i dettagli relativi alla scrittura delle singole istruzioni:
  - codici numerici e formato interno delle istruzioni macchina
  - rappresentazione degli operandi
- gestire direttamente gli indirizzi in memoria per il riferimento ai dati e per i salti al codice

Ad esempio, per scrivere nel linguaggio macchina dell'8086 l'istruzione:

MOV destinazione, sorgente

è necessario scegliere tra:

- **14 diversi codici operativi, in funzione del tipo degli operandi**
- **svariate possibili rappresentazioni degli operandi stessi**

L'istruzione macchina che si ottiene ha una lunghezza variabile tra i **due e i sei byte**, a seconda delle scelte effettuate

Assemblatori e **compilatori** permettono di interfacciare il mondo esterno (i programmi scritti in linguaggi più evoluti) con il livello più basso della macchina

La conoscenza del linguaggio macchina è indispensabile a coloro che scrivono assemblatori e compilatori (in generale, traduttori che generano codice in linguaggio macchina)

Linguaggio Assembler

- insieme di istruzioni di tipo elementare
- costringe ancora il programmatore a ragionare in termini della logica della macchina a cui si riferisce
- **risulta più ad alto livello rispetto al linguaggio macchina**
- nasconde i dettagli realizzativi delle singole istruzione (codici operativi, formati, ecc.)
- associa ai dati e alle istruzioni **nomi simbolici che identificano in modo univoco le corrispondenti posizioni di memoria** (eliminando, così, la necessità di utilizzare indirizzi espliciti)

Istruzioni eseguibili - a cui corrispondono le istruzioni del linguaggio macchina

Direttive (o pseudoistruzioni) - controllano il comportamento dell'assemblatore in fase di traduzione - facilitano lo sviluppo dei programmi - permettono:
- la suddivisione di un'applicazione in più moduli
- il controllo del formato dei listati
- la definizione di macro, ecc.

L'architettura della macchina a cui il linguaggio si riferisce non viene nascosta in alcun modo

Ad esempio, l'assembler dell'8086 tiene conto dell'esistenza dei segmenti e permette di gestirli in modo completo e appropriato

Formato delle istruzioni

[label] istruzione/direttiva [operando/i] [; commento]

- **label** consente di dare un **nome simbolico** (da utilizzare come operando in altre istruzioni) a **variabili di memoria, valori, singole istruzioni, procedure**
- **istruzione/direttiva** è il **mnemonico per un'istruzione eseguibile o una direttiva**: individua il tipo di operazione da eseguire e il numero e il tipo degli operandi
- **operando/i** è una combinazione di nessuna, una o più **costanti**, **riferimenti a registri** o **riferimenti alla memoria**;
  se un'istruzione ammette due operandi:
  - il primo è l'operando destinazione
  - il secondo è l'operando sorgente

  ad esempio, MOV AX, CX copia il contenuto del registro CX nel registro AX
- **commento** consente di rendere più leggibile il programma

E' indifferente l'uso di lettere maiuscole o minuscole

Il tipo di operandi ammessi varia da istruzione a istruzione

Esistono istruzioni che ammettono come operando solo una costante o solo un particolare registro generale

Nomi simbolici

- caratteri ammessi per i nomi simbolici:
  A-Z a-z 0-9 _ $? @
- il primo carattere di un nome non può essere un digit (0-9)
- ogni nome **deve essere univoco** (in genere, all'interno del modulo in cui viene definito)
- **non deve coincidere con una parola riservata** (ad esempio, il nome di un registro o di un operatore)

Costanti numeriche

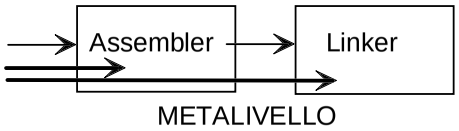
- di default, tutti i valori numerici sono espressi **in base dieci**
- è possibile esprimere le costanti numeriche:
  - **in base 16** (esadecimale) mediante il suffisso **H** (il primo digit deve essere numerico)
  - **in base 8** (octal) mediante il suffisso **O**
  - **in base 2** (binario) mediante il suffisso **B**

Ad esempio:

0FFh	
0h	
777O	
11001B	
FFh	errata !
778O	errata !

DIRETTIVE

Istruzioni di tipo dichiarativo, che forniscono informazioni agli strumenti di sviluppo dei programmi (nel caso del Turbo Assembler, l'assemblatore TASM e il linker TLINK)



Direttive di segmento (standard)

Permettono di definire i vari tipi di segmenti

La definizione di ogni segmento deve iniziare con una **direttiva SEGMENT** e terminare con una **direttiva ENDS**:

```
nomeSeg SEGMENT

contenuto del segmento

nomeSeg ENDS
```


La direttiva **SEGMENT** definisce l'inizio di un segmento

```
nome  SEGMENT  [allineamento] [comb] ['CLASSE']
```

- **nome** - **nome simbolico del segmento** - se è già stato definito un segmento con lo stesso nome, il segmento corrente è la continuazione del precedente

- **allineamento** - controlla l'allineamento in memoria di segmenti con lo stesso nome, definiti in moduli diversi il default è **PARA**

utilizzare:
BYTE per i segmenti che contengono codice
DWORD alla doppia parola
WORD per tutti gli altri segmenti

- **comb** - controlla la modalità di collegamento di segmenti con lo stesso nome, definiti in moduli diversi il default è **PRIVATE**

utilizzare:
STACK per il segmento che contiene lo stack – ciò permette di impostare SS:SP alla fine del segmento stack al momento dell'esecuzione del programma
PUBLIC per tutti gli altri segmenti
AT per segmenti assoluti

- **classe** - controlla l'ordine in cui i segmenti vengono collegati: tutti i segmenti di una determinata classe vengono collocati in un blocco contiguo di memoria, indipendentemente dalla loro posizione nel codice sorgente

utilizzare:
CODE per i segmenti che contengono codice
DATA per i segmenti che contengono dati
STACK per i segmenti che contengono lo stack

La direttiva **ENDS** indica la fine di un segmento

```
nome  ENDS
```

nome è il nome simbolico del segmento

La direttiva **END** indica la fine del programma sorgente

```
END    [startAddress]
```

startAddress, se presente, è un nome simbolico che identifica l'indirizzo di partenza del programma

La direttiva **ASSUME** dice all'assemblatore a quale segmento un determinato registro di segmento sta puntando

```
ASSUME  segReg:nome [,segReg:nome] ...

ASSUME  segReg:NOTHING

ASSUME  NOTHING
```

- **segReg** è il nome di un registro di segmento (CS, SS, DS, ES)
- **nome** è il segmento al quale *l'assemblatore assume* che segReg punti da questo momento in poi (sino a una nuova ASSUME che coinvolga lo stesso registro)
- **NOTHING** cancella l'associazione tra registri e segmenti

L'uso di ASSUME non equivale al caricamento di un registro di segmento,
ma permette all'assemblatore di:
- controllare la validità dei riferimenti alla memoria
- inserire automaticamente eventuali prefissi di segmentazione diversi da quelli di default

Esempio - Programma con tre segmenti (stack, dati e codice):

```
_Stack  SEGMENT  WORD  STACK  'STACK'
        dimensionamento dello stack
_Stack  ENDS

_Data   SEGMENT  WORD  PUBLIC  'DATA'
        definizione dei dati del programma
_Data   ENDS

_Code   SEGMENT  BYTE  PUBLIC  'CODE'
        ASSUME CS:_Code, SS:_Stack
        ASSUME ES:NOTHING
Start:
        MOV  AX, _Data
        MOV  DS, AX
        ASSUME DS:_Data
        . . .
_Code   ENDS

        END  Start
```


Direttive per la definizione di procedure

La definizione di ogni **procedura**
deve iniziare con una **direttiva PROC** e
deve terminare con una **direttiva ENDP**

La direttiva **PROC** definisce l'inizio di una procedura

```
nome  PROC [distanza]
```

- **nome** è il **nome simbolico della procedura** (da utilizzare nelle chiamate alla procedura)
- **distanza** è:
  - **NEAR** - la procedura può essere chiamata solo all'interno del segmento in cui è stata definita (default)
  - **FAR** - la procedura può essere chiamata da qualsiasi segmento

La direttiva **ENDP** indica la fine di una procedura

```
nome  ENDP
```

nome è il nome simbolico della procedura

Ad esempio, per definire la procedura FAR di nome FarProc, si scrive:

```
FarProc    PROC  FAR

           . . .

FarProc    ENDP
```

mentre per definire la procedura NEAR di nome NearProc, si scriverà:

```
NearProc  PROC  NEAR

          . . .

NearProc  ENDP
```


Direttive per la definizione dei dati

Permettono di definire:

- **il nome**
- **il tipo**
- **il contenuto**

delle variabili in memoria

[nome] tipo espressione [, espressione] ...

- **nome** - **nome simbolico del dato**
- **tipo** - **lunghezza del dato** (se scalare) o di ogni elemento del dato (se array) - i tipi più utilizzati sono:

  DB riserva uno o più byte (8 bit)

  DW riserva una o più word (16 bit)

  DD riserva una o più doubleword (32 bit)

- **espressione** - **contenuto iniziale del dato**:
  - un'espressione costante
  - una stringa di caratteri (solo DB)
  - un punto di domanda (nessuna inizializzazione)
  - un'espressione che fornisce un indirizzo (solo DW e DD)
  - un'espressione **DUP**licata

```
ByteVar      DB  0           ;  1 byte inizializzato a 0
ByteArray    DB  1,2,3,4     ;  array di 4 byte
String       DB  '8','0','8','6'  ;  array di 4 caratteri
String       DB  '8086'      ;  equivale al precedente
Titolo       DB  'Titolo',0dh,0ah
                 ;  stringa che contiene anche una
                 ;  coppia di caratteri CR/LF
Zeros        DB  256 dup (0)
                 ;  array di 256 byte inizializzati a 0
Tabella      DB  50 dup (?)
                 ;  array di 50 byte non inizializzati
WordVar      DW  100*50    ;  scalare di una word
Matrix       DW  1,0,0,0,1,0,0,0,1,0,0,0,1,0,0,0
                 ;  array di 16 word
Matrix       DW  4 dup (1, 3 dup (0))
                 ;  equivale al precedente
NearPointer  DW  Matrix
                 ;  contiene l'offset di Matrix
DoubleVar    DD  ? ;  scalare di una doubleword
FarPointer   DD  Matrix
                 ;  contiene l'offset e l'indirizzo del
                 ;  segmento di Matrix
```


La direttiva EQU

Permette di assegnare un nome simbolico a una costante

```
nome  EQU  espressione
```

- **nome** è il **nome simbolico associato alla costante** - può essere utilizzato ovunque al posto della costante stessa - usare lettere maiuscole per distinguere le costanti dalle variabili
- **espressione** deve fornire un valore costante

```
CR      EQU  0dh     ; carattere Carriage Return
LF      EQU  0ah     ; carattere Line Feed
        . . .
Titolo  DB   'Titolo',CR,LF
                     ; stringa che contiene anche
                     ; una coppia di caratteri CR/LF
```

OPERATORI

Permettono di formare espressioni complesse da utilizzare come operandi di istruzioni e direttive

L'assemblatore:

- valuta ogni espressione al momento della traduzione del programma sorgente
- sostituisce il risultato dell'espressione, **che deve essere una costante**, all'espressione stessa

Priorità degli operatori

- gli operatori di priorità maggiore vengono calcolati prima di quelli con priorità inferiore
- gli operatori con uguale priorità vengono calcolati da sinistra a destra
- eventuali sottoespressioni tra parentesi vengono calcolate per prime

Priorità standard

<>, (), [], **LENGTH**, **MASK**, **SIZE**, **WIDTH**

. (selettore di un elemento di struttura)

HIGH, **LOW**

+, - (monadici)

: (override di segmento)

OFFSET, **PTR**, **SEG**, **THIS**, **TYPE**

*, /, **MOD**, **SHL**, **SHR**

+, - (diadici)

EQ, **GE**, **GT**, **LE**, **LT**, **NE**

NOT

AND

OR, **XOR**

LARGE, **SHORT**, **SMALL**, **.TYPE**

Operatore :

Permette di utilizzare un segmento diverso da quello di default

segName **:** espressione

- **segName** è il nome di un registro segmento o di un segmento definito con la direttiva SEGMENT
- **espressione** fornisce un offset relativo al segmento segName

```
MOV   AX, ES:VarX
```

copia in AX il contenuto della variabile VarX appartenente al segmento puntato da ES (il segmento di default è puntato da DS)

```
VarPtr DD      SegData:VarName
```

carica in VarPtr (2 word) l'indirizzo completo (segmento + offset) della variabile VarName appartenente al segmento SegData (che può essere diverso dal segmento di default)

Operatore SEG

Fornisce l'indirizzo del segmento di un'espressione

SEG espressione

espressione è **un riferimento alla memoria** (una label o una variabile)

SEG restituisce l'indirizzo del segmento che contiene il riferimento alla memoria

```
VarX   DB      ?
       . . .
       MOV   AX, SEG VarX
       MOV   DS, AX
```

carica in DS l'indirizzo del segmento che contiene VarX

Operatore OFFSET

Fornisce l'offset di un'espressione all'interno di un segmento

OFFSET espressione

espressione è **un riferimento alla memoria** (una label o una variabile)

OFFSET restituisce il numero di byte dall'inizio del segmento alla locazione della memoria a cui si riferisce espressione

```
VarX   DB      ?
       . . .
       MOV   SI, OFFSET VarX
```

carica in SI l'offset di VarX - non il suo contenuto

MODALITÀ DI INDIRIZZAMENTO DEGLI OPERANDI

L'**operando** di un'istruzione può essere:

- in un **registro**
- nell'**istruzione** stessa - operando immediato
- in **memoria**
- in una **porta di I/O**

L'**indirizzo** di un operando in memoria o in una porta di I/O può essere ottenuto in molti modi diversi

L'**offset** di un operando in memoria viene calcolato dall'EU in funzione della modalità di indirizzamento utilizzata nell'istruzione e viene detto **indirizzo effettivo dell'operando** - **Effective Address** o **EA**

Operando Registro

È l'indirizzamento più **compatto e veloce**

L'operando è già nella CPU e quindi non è necessario accedere alla memoria

Il registro può essere a 8 o a 16 bit

```
MOV  AX, CX          ;  AX <- CX (16 bit)
```

Operando Immediato

L'operando è contenuto nell'istruzione

L'accesso all'operando è **veloce**
l'operando è nella instruction queue

Il dato può essere una costante di 8 o 16 bit

```
MOV  CX, 100         ;  CX <- 100 (16 bit)
```

il valore 100 è memorizzato all'interno dell'istruzione in 2 byte

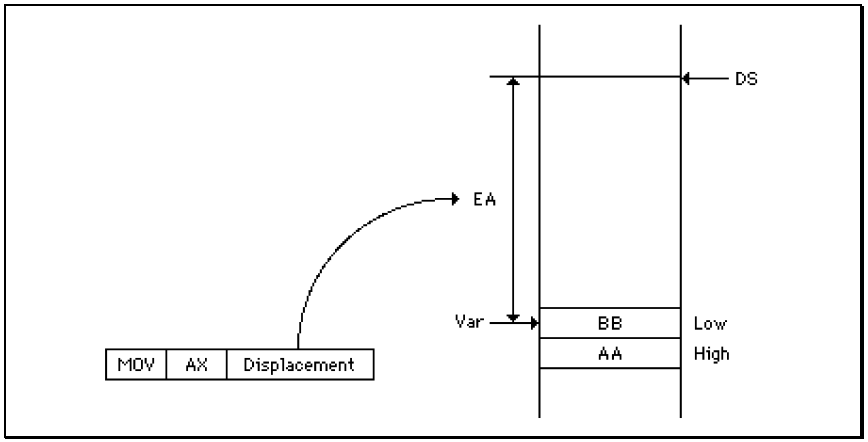
Indirizzamento Diretto

E' l'indirizzamento in memoria più semplice

L'EA è contenuto nel campo displacement dell'istruzione
- un valore senza segno di 8 o 16 bit

```
Var     DW     0AABBh
        . . .
        MOV  AX, Var         ;  AX <- Var (16 bit)

        MOV  AX, [Var]       ;  AX <- Var (16 bit)
```




```
AH  <-  0AAh
AL  <-  0BBh
```

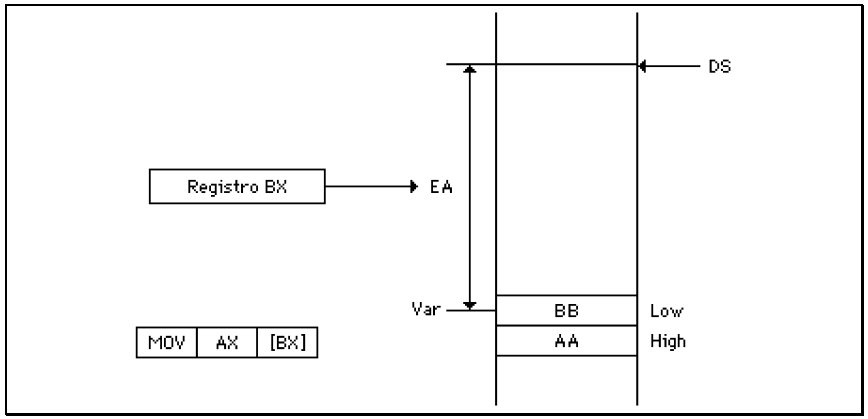
Indirizzamento Indiretto mediante Registro

L'EA si ottiene sommando

- un valore di displacement
- il contenuto corrente di un registro base (BX o BP)
- il contenuto corrente di un registro indice (SI o DI)

tutti i termini tranne uno sono opzionali

```
MOV  BX, OFFSET Var  ;  BX <- offset di Var
MOV  AX, [BX]        ;  AX <- [BX] (16 bit)
```



Se si aggiorna in modo opportuno il contenuto del/dei registro/i, la stessa istruzione può agire su più locazioni di memoria

Attenzione: se si utilizza il registro BP, il segmento di default non è DS, ma SS

```
MOV  AX, [BP+4]          ;  AX <- SS: [BP+4]
MOV  CL, ES:[Var+SI]
MOV  [BX+SI+2], AX
MOV  [BX], 0             ;  8 o 16 bit?
```

Operatore PTR

Cast di un'espressione ad un dato tipo

tipo **PTR** espressione

espressione è un riferimento alla memoria
tipo è **BYTE**, **WORD**, ...

```
MOV   BYTE PTR [BX], 0   ;  8 bit
MOV   WORD PTR [BX], 0   ;  16 bit
```

Esempio di Indirizzamento con Registro Base e Registro Indice

L'EA si ottiene come somma

- di un valore di displacement
- del contenuto di un registro base (BX o BP)
- del contenuto di un registro indice (SI o DI)

Le due componenti dell'indirizzo possono variare al momento dell'esecuzione

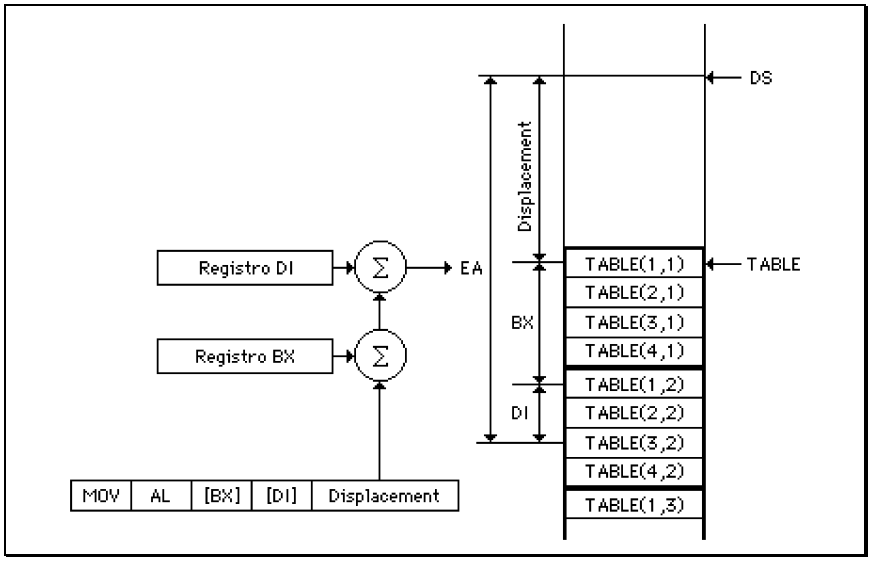
Questa caso è utile per accedere, ad esempio, ai diversi elementi di una matrice: il displacement fornisce l'indirizzo di partenza dell'array, mentre il registro base e il registro indice selezionano uno degli elementi (il primo elemento viene selezionato quando entrambi i registri valgono 0).

TABLE è l'indirizzo simbolico di una matrice di byte 4x4, memorizzata per colonne nel data segment corrente

```
MOV BX, 4                  ;  inizializza BX a 4
MOV DI, 2                  ;  inizializza DI a 2
MOV AL, [TABLE+BX+DI] ;  AL <- TABLE(3,2)
```

BX contiene il displacement tra l'indirizzo di partenza della matrice e la colonna selezionata (la seconda)

DI contiene il displacement tra l'indirizzo di partenza della colonna selezionata e la riga selezionata (la terza)



Indirizzamento delle porte di I/O

Per accedere ad una **porta di I/O mappata in memoria**, si utilizza una qualsiasi modalità di indirizzamento alla memoria

Per accedere ad una **porta di I/O NON mappata in memoria**, è necessario utilizzare le istruzioni:

IN accumulatore, porta

OUT porta, accumulatore

IN (INput) trasferisce un dato da una porta di input, al registro AX o AL

OUT (OUTput) trasferisce un dato dal registro AX o AL ad una porta di output

La porta è identificata:
- da un valore costante compreso tra 0 e 255
  indirizzamento diretto
- dal contenuto del registro DX (tra 0 e 65.535)
  indirizzamento indiretto

```
IN  AX, 15     ;  AX <- port15        (word)
OUT  3, AL     ;  port3 <- AL         (byte)
IN  AX, DX     ;  AX <- port[DX]      (word)
```


L'operatore PTR

Cast di un'espressione ad un dato tipo

tipo **PTR** espressione

espressione è **un riferimento alla memoria**
tipo è:
- se dati: **byte**, **word**, ...
- se codice: **near**, **far**, ...

```
MOV   [BX], 0                ;  ?
MOV   BYTE PTR [BX], 0       ;  8 bit
MOV   WORD PTR [BX], 0       ;  16 bit


CALL  [BX]                   ;  ?
CALL  WORD  PTR  [BX]        ;  Near call
                             ;  BX -> offset
CALL  DWORD  PTR  [BX]       ;  Far call
                             ;  BX -> seg:offset
```

L'operatore SHORT

Informa l'assemblatore che espressione è distante dalla locazione corrente
da -128 a +127 byte ***(indirizzo di un byte)***

SHORT espressione

espressione è **un riferimento a una label nel segmento codice corrente**

```
    JMP    SHORT  Lab1          ;  riferimento in avanti
    . . .
Lab1:
```

La direttiva LABEL

forza un simbolo ad un tipo specificato

nome LABEL tipo

```
words       LABEL  WORD         ;  struttura come word
bytes       db  howmany DUP (?)
    . . .
    MOV    AX, words            ;  prima word
```


Direttiva PUBLIC

Rende accessibili da altri moduli uno o più simboli

PUBLIC symbol [, symbol ...]

symbol è il nome di una variabile o di una procedura

```
_DATA       SEGMENT  WORD PUBLIC 'DATA'
ARR_SIZE    EQU      100
MemVar      DW       10
MemArr      DB       ARR_SIZE  DUP (0)
            . . .
_DATA       ENDS
_CODE       SEGMENT  BYTE PUBLIC 'CODE'
NearProc    PROC
            . . .
FarProc     PROC     FAR
            . . .
_CODE       ENDS
            PUBLIC MemVar,MemArr
            PUBLIC NearProc,FarProc

            END
```

Direttiva EXTRN

Rende accessibili uno o più simboli definiti in un altro modulo

```
EXTRN  nome:tipo [, nome:tipo ...]
```

nome è il nome di un simbolo definito PUBLIC in un altro modulo
tipo è il tipo del simbolo:

- BYTE, WORD, ...
- NEAR, FAR, ...

```
EXTRN    MemVar:WORD,MemArr:BYTE
EXTRN    NearProc:NEAR,FarProc:FAR
. . .
MOV      AX, MemVar
MOV      BX, OFFSET MemArr
. . .
CALL     NearProc
CALL     FarProc
```


ATTENZIONE

Attenzione alla corrispondenza dei tipi

```
          PUBLIC   FarProc
FarProc   PROC     FAR
          . . .
          RET
FarProc   ENDP
```

```
          EXTRN    FarProc:Near ; ERRORE!
          . . .
          CALL     FarProc
          . . .
```

Attenzione alle maiuscole/minuscole

Usare i qualificatori **/ml** o **/mx** insieme a TASM

PROCEDURE E FUNZIONI

```
myprog1 segment  public  'CODE'
nome proc
        CALL proced
       ...
nome endp
myprog1 ends

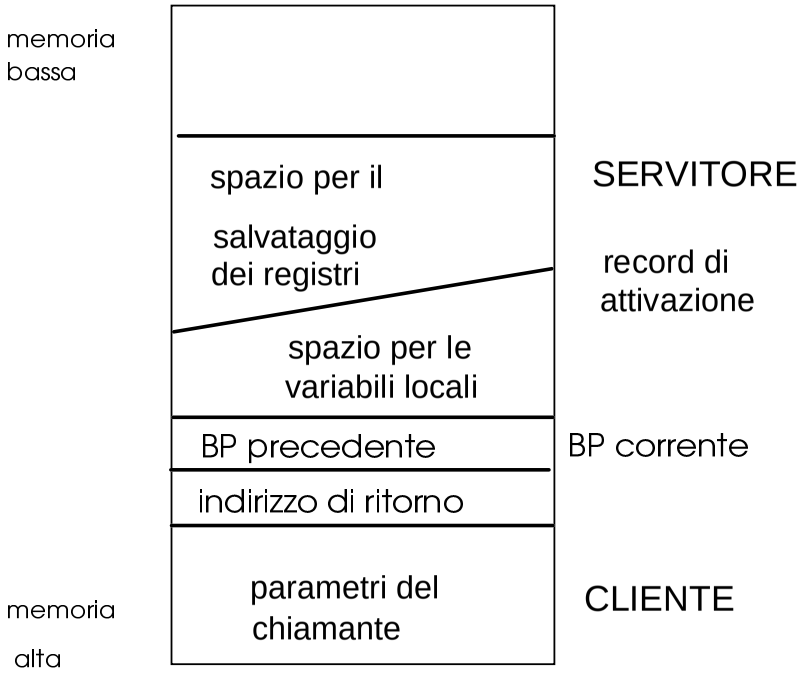
myprog2 segment  public  'CODE'
proced  proc FAR
            ...
            RET
proced  endp
myprog2 ends
```

USO dello STACK

1) per la valutazione di **espressioni aritmetiche**
2) per **procedure innestate** (**ricorsive**)
   la CALL inserisce nello stack l'indirizzo di ritorno;
   la RET estrae ordinatamente e lo ripristina.
3) passaggio dei **parametri**
   passati per **valore**
   o per **indirizzo**
4) **variabili locali**

STACK frame

Possibile uso dello stack
Ogni procedura riserva il proprio **record di attivazione** separato da quello del chiamante



Protocollo ***chiamante/chiamato***

Il **programma cliente** (chiamante) deve eseguire:

```
        PUSH par1
        PUSH par2
        CALL procedura
        POP uscita ; parametro di uscita sullo stack
```

La **procedura chiamata** (**servitore**)

```
procedura PROC far
        PUSH BP
        MOV BP,SP          ;  un nuovo frame
        SUB SP,10          ; spazio per le variabili locali
              ; usate come [BP - i] con i = 2, 4, 6,...
        PUSH AX            ; salvataggio registri cliente
        PUSH BX            ; che il chiamato utilizza
        MOV AX, [BP+6] ; recupero dei parametri
        MOV BX, [BP+8] ; il primo parametro
   < fase di elaborazione ed esecuzione  >
        MOV [BP+8], parametrouscita
        ; il parametro di uscita ricopre l'ingresso
        POP BX ;  ripristino dei valori nei registri
        POP AX ;
        MOV SP,BP ; recupero variabili locali
        POP BP       ; si ristabilisce il vecchio frame
        RET 2        ; eliminazione parametro ingresso
procedura ENDP
```


Gerarchia per le interruzioni, con priorità decrescente:

- interrupt interni (o di errore)
- interrupt non mascherabili
- interrupt mascherabili esterni
- interrupt di single-step.

Al verificarsi di più interruzioni **"insieme"**
===>

priorità

Routine di interruzione **brevi**
od almeno
tempo di disabilitazione
delle interruzioni esterne
deve essere limitato al minimo

Protocollo di uso dello stack per interruzioni
analogo a quello per le procedure
===>
per garantire innestamento corretto

SCHEMA DI UN PROGRAMMA ASSEMBLER

in FILE1.asm

```
code1 segment 'CODE'
  assume cs: code1, ds: data1, ss:stack1
 proced1 proc far
 ...
 proced1 endp
 proced2 proc near
 ...
 proced2 endp
code1 ends

code2 segment 'CODE'
 assume cs: code2, ds: data1, ss:stack1
 proced3 proc far
 ...
 proced3 endp
code2 ends
data1 segment 'DATA'
 .... ; definizione dei dati globali
data1 ends

stack1 segment stack 'STACK'
 ...  ; definizione del segmento stack
stack1 ends
end
```


ESEMPIO di funzione ricorsiva

- sviluppata in un unico file;

Il sistema operativo fornisce le funzioni di input/output

```
function f91 (x:integer):integer;
 begin    if x > 100 then  f91 := x - 10
          else  f91 := f91 (f91( x + 11))
          endif;
 end  f91;
```

Input di un unico carattere

funzione di sistema operativo (**BIOS**):

valore fornito con **<alt><cifre dalla tastiera numerica>**

Output di caratteri

stampa degli ascii corrispondenti

La stringa (inversa) di caratteri ascii usando funzioni di sistema (**DOS**)

```
code1  segment para public 'CODE'
prog   proc far
       assume      cs:code1,ds:code1,ss:stack1
       push  ds    ; salvataggio dell'indirizzo di
       sub   ax,ax ; ritorno al sistema operativo
       push  ax            ; all'indirizzo di offset 0
                   ; del programma
; il programma è visto come una procedura del
; sistema operativo: si termina con una RET
```

```
        mov     ax,0
        int     16h
        ; attesa di un carattere da console
        mov     ah,0
        push    ax; il parametro di input nello stack
        call    f91     ;procedure vicine
                        ;(cs non variato)
        pop     ax
        call    bintoasci; converti e stampa
        ret
prog    endp

f91     proc    near  ; nuovo frame sullo stack
        push    bp
        mov     bp,sp  ;
        push    ax      ; salvataggio valore di ax
        mov     ax, [bp+4]
                ; recupero del valore del parametro
        cmp     ax,100
        jbe     ricorri  ; se inferiore, ricorri
        sub     ax,10
        mov     [bp+4], ax ; valore di ritorno
        pop     ax
        pop     bp
        ret     ; parametro di ritorno sullo stack
ricorri:        ; invocazione ricorsiva
```



```
        add     ax,11  ; calcolo di  f91 ( x + 11)
        push    ax
        call    f91
        pop     ax                      ; calcolo di f91 (f91
        (x + 11))
        push    ax
        call    f91
        pop     ax                      ; valore di ritorno
        sullo stack
        mov     [bp+4], ax
        pop     ax
        pop     bp
        ret
f91     endp

bintoasci       proc  ; non crea un nuovo stack frame
 comment* stampa gli ascii per il binario in AX
            in ordine inverso + cr e lf
          *
        push    dx   ; salvataggio registri utilizzati
        push    si
        push    ax
        mov     si,10  ; divisioni successive
again: sub      dx,dx    ; i caratteri sono scritti
        div     si        ; in ordine inverso
        add     dx,'0'            ; la cifra di resto in ascii
        call    stampa            ; output di un carattere
        or      ax,ax
        jnz     again
```

```
        mov     dl,  0dh ; stampa <cr>
        call    stampa
        mov     dl,  0ah ; stampa <lf>
        call    stampa
        pop     ax
        pop     si
        pop     dx
        ret
bintoasci endp

stampa          proc    ; stampa di un carattere
                        ; il carattere è in dl (DX)
        push    ax   ; salvataggio valore registro AX
        push    dx   ; e DX
        mov     ah,2            ; carattere in dl
        int     21h             ; interrupt al DOS
        pop     dx
        pop     ax
        ret
stampa          endp
code  ends

stack1segment 'STACK'
        db      100 dup('stack   ')
stack1ends  ; segmento di stack
        end
```


PROTOCOLLO CLIENTE/SERVITORE

Passaggio di parametri

❶ mediante registri

```
MOV   CX, 5
MOV   DL, 'A'
CALL  display  ;  visualizza "AAAAA"
```

❷ mediante stack (**convenzione C**)

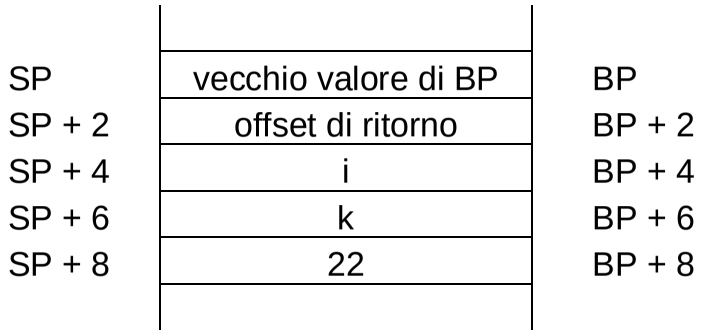
```
func(i,k,22)

MOV   AX, 22
PUSH  AX
PUSH  k
PUSH  i
CALL  _func
ADD   SP, 6
```

Accesso ai parametri passati sullo stack
Chiamata NEAR

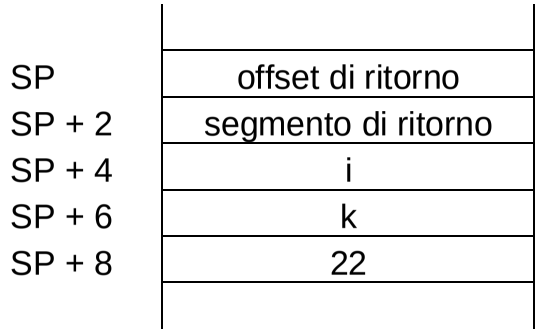



```
_func  PROC NEAR
       PUSH BP
       MOV  BP, SP
```

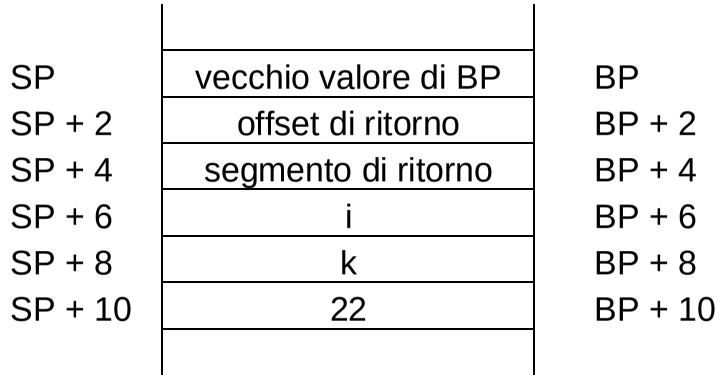



```
       MOV  AX, [BP+4]    ; primo argomento
       ADD  AX, [BP+6]    ; secondo argomento
       SUB  AX, [BP+8]    ; terzo argomento
       POP  BP
       RET
_func  ENDP
```


Chiamata FAR




```
_func  PROC FAR
       PUSH BP
       MOV  BP, SP
```




```
       MOV  AX, [BP+6]    ; primo argomento
       ADD  AX, [BP+8]    ; secondo argomento
       SUB  AX, [BP+10]   ; terzo argomento
       POP  BP
       RET
_func  ENDP
```

Variabili locali sullo stack

```
_func  PROC NEAR
       PUSH BP
       MOV  BP, SP
;
;      Servono 2 variabili locali di tipo "short int"
;
       SUB  SP, 4   ; riserva 4 byte sullo stack
```

SP	2° v ariabile locale	BP – 4
SP + 2	1° v ariabile locale	BP – 2
SP + 4	vecchio valore di BP	BP
SP + 6	offset di ritorno	BP + 2
SP + 8	1° argomento	BP + 4
SP + 10	2° argomento	BP + 6
SP + 12	3° argomento	BP + 8

```
       MOV  WORD PTR [BP-2], 0
       MOV  WORD PTR [BP-4], 0
       . . .
       MOV  SP, BP
       POP  BP
       RET
_func  ENDP
```


Esempio

```
typedef struct
        {
        int giorno,mese,anno;
        } DataType;

void fun(DataType *dpar)
        {
        DataType dtemp;
        dtemp.giorno = dpar->giorno;
        dtemp.mese = dpar->mese;
        dtemp.anno = dpar->anno;
        }


_fun proc   near
     push   bp
     mov    bp,sp
     sub    sp,6              ; 6 = sizeof(DataType)
     push   si
     mov    si,word ptr [bp+4]   ; dpar
     mov    ax,word ptr [si]     ; ax <- dpar->giorno
     mov    word ptr [bp-6],ax   ; dtemp.giorno <- ax
     mov    ax,word ptr [si+2]   ; ax <- dpar->mese
     mov    word ptr [bp-4],ax   ; dtemp.mese <- ax
     mov    ax,word ptr [si+4]   ; ax <- dpar->anno
     mov    word ptr [bp-2],ax   ; dtemp.anno<- ax
     pop    si
     mov    sp,bp
     pop    bp
     ret
_fun endp
```

Schema Generale
di una procedura assembler
(convenzione C)

```
_fun proc    near (o far)
     push    bp
     mov     bp,sp
;
;    allocazione spazio su stack per variabili locali
;
     sub     sp,xxx
;
;    eventuale salvataggio registri
;    in uscita devono avere il vecchio valore:
;            BP, SP, CS, DS, SS, DI, SI
;    in uscita possono non avere il vecchio valore:
;            AX, BX, CX, DX, ES e il registro flag
;
     push    di; se utilizzato
     push    si; se utilizzato
;
;    procedura
;
     . . .
;
;    ripristino registri
;
     pop     si
     pop     di
     mov     sp, bp     ; ripristino sp
     pop     bp         ; ripristino bp
     ret
_fun endp
```


Restituzione di valori

In AX vengono restituiti i valori a 8 e a 16 bit:

[unsigned] char
[unsigned] int
near *

```
int fun1()
        {. . .
        return 1;
        }

        mov     ax,1
```

In DX:AX vengono restituiti i valori a 32 bit:

[unsigned] long
far *

```
long fun2()
        { . . .
        return 1;
        }

        xor     dx,dx
        mov     ax,1
```

E per parametri di dimensioni maggiori?
vedi restituzione di un record C

Passaggio di parametri

❸ mediante stack (**convenzione Pascal**)

```
        func(i,k,22)

        PUSH i
        PUSH k
        MOV  AX, 22
        PUSH AX
        CALL FUNC          ; maiuscolo senza _
;       ADD  SP, 6         non esiste più !
```

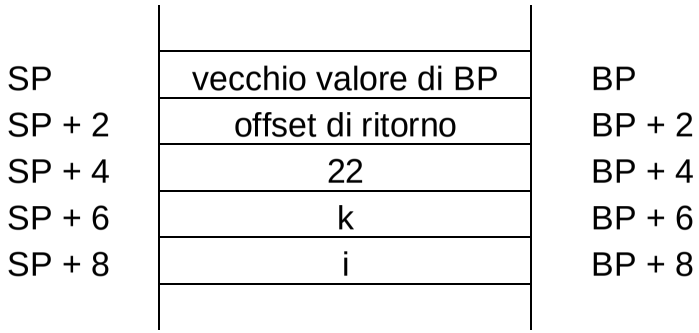
Il chiamante non deve estrarre gli argomenti
che devono essere estratti dal chiamato

Il chiamato deve sapere quanti parametri estrarre

Accesso ai parametri passati sullo stack
Chiamata NEAR (**convenzione Pascal**)




```
func    PROC NEAR
        PUSH BP
        MOV  BP, SP
```




```
        MOV  AX, [BP+8]    ;  primo argomento
        ADD  AX, [BP+6]    ;  secondo argomento
        SUB  AX, [BP+4]    ;  terzo argomento
        POP  BP
        RET  6             ;  ret + add sp,6
func    ENDP
```

MODELLI DI MEMORIA

Accesso contemporaneo a 4 segmenti di 64k byte

1 segmento codice - istruzione da eseguire in CS:IP
1 segmento stack - top dello stack in SS:SP
2 segmenti per i dati del programma - DS e ES

Programmazione in un linguaggio ad alto livello
si sceglie una delle configurazione di segmenti (**modelli di memoria**) fornite dal compilatore

6 modelli di memoria di complessità crescente:
- modello **tiny**
  1 solo segmento per codice, dati, stack e heap
- modello **small**
  1 segmento codice, 1 segmento per dati, stack e heap
- modello **compact**
  1 segmento codice, 1 segmento dati, 1 segmento per lo stack e più segmenti per l'heap
- modello **medium**
- modello **large**
- modello **huge**
  più segmenti codice e più segmenti dati

Scelta in funzione:
- della dimensione del codice
- della dimensione dei dati

Programmazione in assembler
si può scegliere una qualsiasi configurazione di segmenti, utilizzando in modo opportuno la direttiva SEGMENT

Programmazione in
assembler + linguaggio ad alto livello
il codice assembler deve utilizzare lo stesso modello di memoria scelto per compilare il codice ad alto livello

Esempio utilizzato nel seguito: 2 moduli: P.C e S.ASM

Modulo P.C:

```
void main()
  {  printf("%d\n",lung("Modelli di memoria!"));  }
```

Modulo S.ASM - sottoprogramma e dati statici - il suo contenuto varia al variare del modello di memoria
può essere espresso in C nel seguente modo:

```
static char cc = ',';/*  variabili statiche  */
static int j;            /*  per esemplificazione  */

lung(char *str)
  {
  if(*str == cc) return 0;
  for (j = 0; *str++; j++) ;
  return j;
  }
```

Segmento **stack** ed eventuali segmenti che compongono l'**heap** sono automaticamente definiti e gestiti a livello superiore (C e DOS)

In alcuni modelli di memoria, segmento dati diviso in:

segmento _DATA - contiene le variabili cui è stato assegnato un valore iniziale (variabile cc)

segmento _BSS - contiene le variabili che non sono state espressamente inizializzate (variabile j) - il suo contenuto viene automaticamente posto a 0 dal loader

Direttiva GROUP

Permette di riunire un insieme di segmenti in un unico segmento, accessibile mediante un unico registro di segmento:

nome GROUP $nome_1$ [, $nome_2$ ...]

dove:
- **nome** è il nome simbolico del gruppo di segmenti
- **$nome_1$**, **$nome_2$**, ... sono i nomi dei segmenti che devono essere uniti

Esempio - Per riunire segmento dati e segmento stack in un unico segmento di nome DGROUP:

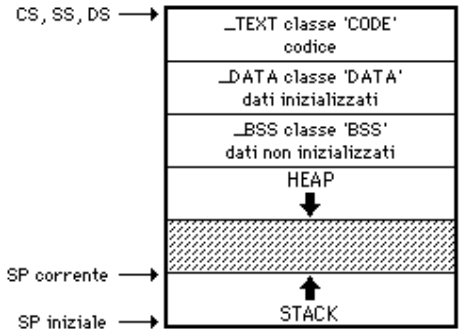
```
DGROUP GROUP _Data,_Stack

_Data   SEGMENT WORD PUBLIC 'DATA'
        definizione dei dati del programma
_Data   ENDS

_Stack  SEGMENT WORD STACK 'STACK'
        dimensionamento dello stack
_Stack  ENDS

_Code   SEGMENT BYTE PUBLIC 'CODE'
        ASSUME
        CS:_Code,SS:DGROUP,ES:NOTHING
Start:
        MOV AX,DGROUP
        MOV DS,AX
        ASSUME DS:DGROUP
        codice
_Code   ENDS
```

Modello di memoria TINY



Riferimenti al codice e ai dati mediante il solo **offset** nel segmento - **puntatori near**

Contenuto di S.ASM:

```
DGROUP group  _DATA,_BSS

_DATA    segment word public 'DATA'
cc       db       ','
_DATA    ends

_BSS     segment word public 'BSS'
j        dw       ?
_BSS     ends

_TEXT    segment byte public 'CODE'
         assume cs:_TEXT,ds:DGROUP
```



```
_lung    proc   near
         push   bp
         mov    bp,sp
         push   si
         mov    si,word ptr [bp+4]      ; si <- str

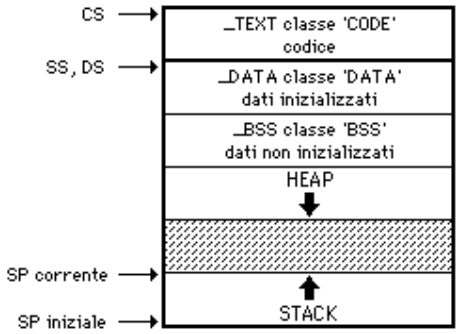
;        if(*str == cc) return 0;

         mov    al,byte ptr [si]        ; al <- *str
         cmp    al,byte ptr DGROUP:cc
         jne    for
         xor    ax,ax                   ; return 0
         jmp    short fine

;        for (j = 0; *str++; j++) ;

for:     mov    word ptr DGROUP:j,0
cicla:   mov    bx,si
         inc    si                      ; str++
         cmp    byte ptr [bx],0
         jz     fine1
         inc    word ptr DGROUP:j
         jmp    cicla
fine1:   mov    ax,word ptr DGROUP:j    ; return j
fine:    pop    si
         pop    bp
         ret
_lung    endp
_TEXT    ends
         public _lung   ; visibilità cc e j limitata
         end
```

Modello di memoria SMALL

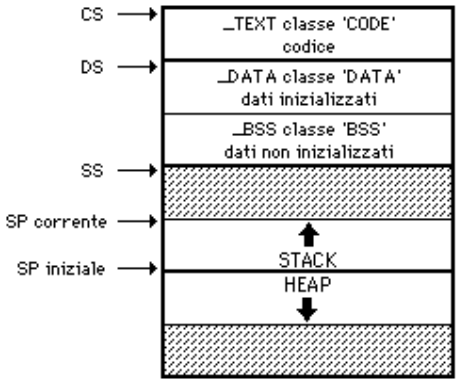


Riferimenti al codice e ai dati mediante il solo **offset** nel corrispondente segmento - **puntatori near**

Contenuto di S.ASM:

identico a quello relativo al modello TINY

Modello di memoria COMPACT



Riferimenti al codice mediante il solo offset nel corrispondente segmento - **puntatori near**

Riferimenti ai dati mediante **puntatori far** (segmento:offset) - occorre gestire in modo uniforme riferimenti a dati che possono essere memorizzati:

- nel segmento dati
- nel segmento stack
- in uno dei segmenti che compongono l'heap

Contenuto di S.ASM:

```
            . . .
_lung       proc    near
            push    bp
            mov     bp,sp
            push    si
            les     si,dword ptr [bp+4]  ; es:si <- str

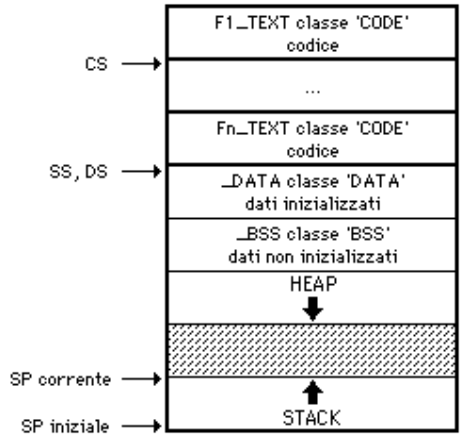
;           if(*str == cc) return 0;

            mov     al,byte ptr es:[si]      ; al <- *str
            cmp     al,byte ptr DGROUP:cc
            jne     for
            xor     ax,ax                    ; return 0
            jmp     short fine

;           for (j = 0; *str++; j++) ;

for:        mov     word ptr DGROUP:j,0
cicla:      mov     bx,si
            inc     si                       ; str++
            cmp     byte ptr es:[bx],0
            jz      fine1
            inc     word ptr DGROUP:j
            jmp     cicla
fine1:      mov     ax,word ptr DGROUP:j     ; return j
fine:       pop     si
            pop     bp
            ret
_lung       endp
            . . .
```


Modello di memoria MEDIUM



Riferimenti al codice mediante mediante **puntatori far**

Riferimenti ai dati mediante **puntatori near**

Il nome di ogni segmento codice è formato dal nome del file contenente il codice (F1, ..., Fn), seguito da _TEXT
Il registro CS punta sempre al segmento che contiene il codice correntemente in esecuzione

Contenuto di S.ASM:

```
            . . .
S_TEXT      segment byte public 'CODE'
            assume cs:S_TEXT,ds:DGROUP
```

```
_lung     proc   far
          push   bp
          mov    bp,sp
          push   si
          mov    si,word ptr [bp+6]     ; si <- str

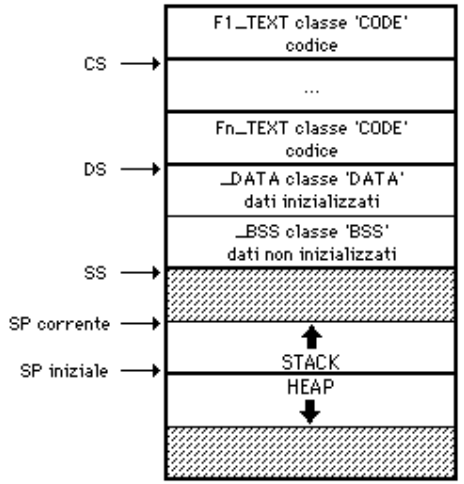
;         if(*str == cc) return 0;

          mov    al,byte ptr [si]       ; al <- *str
          cmp    al,byte ptr DGROUP:cc
          jne    for
          xor    ax,ax                  ; return 0
          jmp    short fine

;         for (j = 0; *str++; j++) ;

for:      mov    word ptr DGROUP:j,0
cicla:    mov    bx,si
          inc    si                     ; str++
          cmp    byte ptr [bx],0
          jz     fine1
          inc    word ptr DGROUP:j
          jmp    cicla
fine1:    mov    ax,word ptr DGROUP:j   ; return j
fine:     pop    si
          pop    bp
          ret
_lung     endp
S_TEXT    ends
          public _lung
          end
```


Modello di memoria LARGE

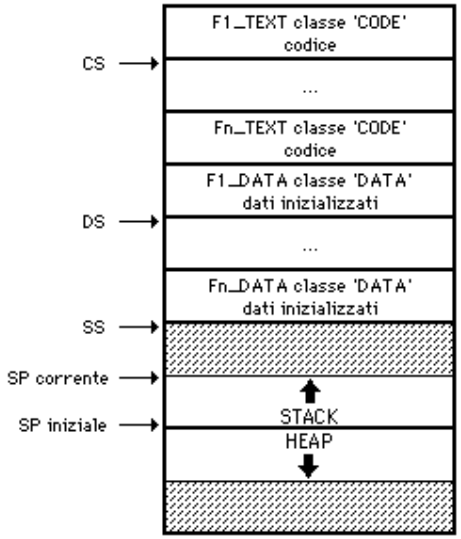


Riferimenti al codice e riferimenti ai dati mediante **puntatori far**

Il contenuto di S.ASM coincide con:

- quello relativo al **modello medium** per quanto riguarda il **codice** - procedura far e segmento di nome S_TEXT
- quello relativo al **modello compact** per quanto riguarda i **dati** - riferimenti far e override di segmento

Modello di memoria HUGE



Riferimenti al codice e riferimenti ai dati mediante **puntatori far**

Il nome di ogni segmento dati è formato dal nome del file contenente il codice (F1, ..., Fn), seguito da _DATA
Il registro DS **deve puntare** al segmento che contiene i dati corrispondenti al codice in esecuzione

Contenuto di S.ASM:

unico segmento **S_DATA** contenente dati **inizializzati**
assenza del segmento _BSS

```
S_DATA  segment word public 'DATA'
cc      db      ','
        db      0       ;   allinea j alla word
j       dw      0
S_DATA  ends

S_TEXT  segment byte public 'CODE'
        assume cs:S_TEXT,ds:S_DATA
_lung   proc    far
        push    bp
        mov     bp,sp
        push    ds
        mov     ax,S_DATA
        mov     ds,ax
        push    si
        les     si,dword ptr [bp+6]     ; es:si <- str
        . . .
fine:   pop     si
        pop     ds
        pop     bp
        ret
_lung   endp
S_TEXT  ends
        public  _lung
        end
```

Riepilogo

Modello di memoria	Riferimenti al codice	Riferimenti ai dati	Dimensione applicazione
Tiny	near	near	codice + dati <= 64k
Small	near	near	codice <= 64k + dati <= 64k
Compact	near	far	codice • 64k + dati <= 1M
Medium	far	near	codice <= 1M + dati <= 64k
Large	far	far	codice + dati <= 1M
Huge	far	far	codice + dati <= 1M

SVILUPPO DI UN PROGRAMMA

- **COMPILATORE**
  traduce un modulo sorgente (file) scritto in linguaggio ad alto livello (ad es. C, Pascal, ...) in codice rilocabile ma non eseguibile (modulo oggetto)

- **ASSEMBLATORE**
  traduce un modulo sorgente (file) scritto in linguaggio Assembler in codice rilocabile ma non eseguibile (modulo oggetto)

- **LINKER** (Collegatore)
  collega tra loro i vari moduli oggetto e crea un modulo eseguibile

- **LOADER** (Caricatore)
  carica in memoria il modulo eseguibile

- **DEBUGGER**
  controlla l'esecuzione del programma in fase di messa a punto

COMPILATORE

FASE DI ANALISI:

- **analisi lessicale** (scanner) - esamina i caratteri del sorgente e genera i simboli (atomici) del programma (identificatori, parole riservate, ecc.)
- **analisi sintattica** (parser) - stabilisce se il programma è sintatticamente corretto
- **analisi semantica** - stabilisce se il programma è semanticamente corretto

  {int x; char x[4]; ...

  sintatticamente corretto

  semanticamente errato

FASE DI SINTESI:

- **preparazione alla generazione del codice**:
  - **ottimizzazione** rispetto:
    - l'occupazione di memoria
    - il tempo di esecuzione
- **generazione del codice** (in forma rilocabile)
- eventuale **ottimizzazione** finale del codice (peephole optimization)

  add ax, 1 4 clock, 4 byte

  inc ax 3 clock, 1 byte - CF inalterato

TAVOLE DI INFORMAZIONE:

- contengono tutte le informazioni ottenute durante la fase di analisi:
  - dichiarazioni di procedure
  - dichiarazioni di variabili
  - cicli
  - ecc.
- il numero di tavole dipende:
  - tipo di linguaggio sorgente
  - tipo di codice oggetto da generare
  - grado di complessità del compilatore

TAVOLA DEI SIMBOLI (Symbol Table)

di ogni identificatore che compare nel modulo sorgente contiene:

- il nome simbolico
- il tipo (procedura, variabile intera, array di reali, label, ...)
- la visibilità (statico, pubblico, esterno, ...)
- la posizione (offset) nel segmento che lo contiene
- ecc.

ASSEMBLATORE

Può essere a 1 o più passi:
- ad ogni passo viene effettuata una scansione del programma sorgente
- in genere a 2 passi
- TASM è a 1 passo (default)

1 PASSO (analisi):
- riconoscimento delle **macro** e loro espansione
- riconoscimento delle **direttive**
- riconoscimento delle **istruzioni** ammesse
- creazione della **Symbol Table** - aggiornamento di un contatore per ogni segmento

2 PASSO (sintesi):
- **traduzione** del programma (utilizzo della tabella contenente i codici operativi)
- sostituzione degli **operandi simbolici** con i corrispondenti indirizzi (rilocabili)
- segnalazione di identificatori non definiti
- generazione **listato**
- generazione **istruzioni speciali per il linker** - simboli pubblici o esterni

IL MODULO OGGETTO

Ogni modulo oggetto contiene:
- informazioni necessarie al **collegamento con altri moduli** (anche librerie di sistema, non interrupt software) - ESD (External Symbol Dictionary)
- informazioni necessarie alla **rilocazione**
- istruzioni macchina e dati generati dalla traduzione del sorgente

Modulo X.ASM

```
C segment byte public 'CODE'
A1          proc
            . . .
            call  B1
            . . .
            public  A1
            extrn  B1:near
```

Modulo Y.ASM

```
C segment byte public 'CODE'
B1          proc
            . . .
            call  A1
            . . .
            public  B1
            extrn  A1:near
```

Modulo X.OBJ

A1	proc	public	C:0
B1	proc	extrn	?

Informazioni sugli oggetti che possono dover essere rilocati

Istruzioni e dati inizializzati

Modulo Y.OBJ

A1	proc	extrn	?
B1	proc	public	C:0

Informazioni sugli oggetti che possono dover essere rilocati

Istruzioni e dati inizializzati

LINKER

I compiti del linker sono:

- **unificazione dei segmenti** che devono essere riuniti (stesso nome, group, ...) ed eventuale rilocazione degli oggetti ad essi appartenenti
- **risoluzione dei simboli esterni**

Il linker collega anche eventuali **librerie esterne** (si pensi alle librerie C)

Il risultato è un **modulo eseguibile** che contiene:

- codice e dati **rilocabili**
- direttive per il loader

Modulo X.OBJ

A1	proc	public	C:0
B1	proc	extrn	?

Informazioni sugli oggetti che possono dover essere rilocati

Istruzioni e dati inizializzati

Modulo Y.OBJ

A1	proc	extrn	?
B1	proc	public	C:0

Informazioni sugli oggetti che possono dover essere rilocati

Istruzioni e dati inizializzati

RILOCAZIONE in momenti diversi:

- *a livello sorgente*
- *durante la traslazione*
- *dopo la traslazione e prima del caricamento*
- al **caricamento**
- dopo il caricamento e prima della esecuzione
- durante la esecuzione

CARICAMENTO

La fase di caricamento può prevedere

rilocazione STATICA

Ad esempio, si trovano i valori per i simboli di segmento prima della esecuzione ed al caricamento

DEBUGGING

Debugger più semplice ==>
carica il programma come sottoprogramma
visualizza ed **altera** il contenuto della memoria

L'esecuzione del programma viene **controllata:**

- attraverso il flag di **single-step (TF)**
- i breakpoint via interruzione **single-byte**
  il vecchio valore del byte deve essere salvato
  Alla interruzione il codice viene ripristinato.
- alla **terminazione** del programma, una routine del debugger ripresenta ancora l'ambiente di debugging.

RILOCAZIONE DINAMICA

```
MAINDATA  SEGMENT 'DATA'
oldptr          DD program   ; punta al codice corrente
newptr          DD target
pr_size         DW program_length
MAINDATA  ENDS

SOURCE  SEGMENT 'CODE'
; posizione iniziale codice
assume CS: SOURCE, ...
    program  PROC far
                        ...
program_length EQU
        (OFFSET THIS BYTE) - (OFFSET program)
    program endp
SOURCE  ENDS

TARGET  SEGMENT 'DATA'
; posizione finale codice
            DB program_length DUP (?)
TARGET  ENDS
```



```
MAINCODE SEGMENT 'CODE'
ASSUME CS:MAINDATA, DS: MAINDATA, SS:..
main   proc far
<carica i registri di segmento e chiama la procedura >
            CALL DWORD PTR oldptr ; chiamata indiretta
<muovi dal source al destination>
            MOV         CX, program_length
            PUSH        DS
            LES         DI, newptr
            LDS         SI, oldptr
            CLD
            REP         MOVSB; copia il bloccco
<   cancella il vecchio segmento e
    sistema gli indirizzi in maindata >
            MOV         AX, DS
            POP         DS
            MOV         WORD PTR oldptr+2, ES
            SUB         DI, pr_size;  aggiusta DI
            MOV         WORD PTR oldptr, DI
            MOV         WORD PTR newptr+2, AX
            SUB         SI,pr_ size; aggiusta SI
            MOV         WORD PTR newptr, DI
< nuova chiamata >
            CALL        DWORD PTR oldptr
            HLT
main   ENDP
MAINCODE ENDS
END main
```

C ⇔ Assembler

C:
- scelta del modello di memoria
- creazione dei file .OBJ

Assembler:
- utilizzo di una struttura dei segmenti omogenea con il modello di memoria del C
- utilizzo di nomi simbolici adeguati per gli external (underline, /ml, /mx)
- utilizzo delle convenzioni C per:
  - passaggio degli argomenti alla procedura chiamata
  - salvataggio dei registri
  - restituzione del risultato al chiamante
- creazione dei file .OBJ

Link:
- collegamento dei moduli oggetto creati da C e da Assembler

Attenzione - main sempre in C

ORGANIZZAZIONE INTERNA MS-DOS

Sistema a livelli

UTENTE	**PROGRAMMA**
Linguaggio comandi **Job Control Language** (file COMMAND.COM)	**LINGUAGGI di PROGRAMMAZIONE**

SISTEMA OPERATIVO
DOS file system
(file IBMDOS.COM)

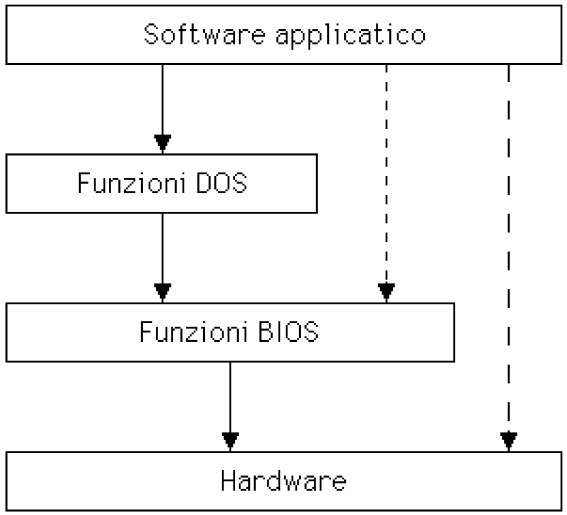
BIOS gestione diretta dispositivi
(file IBMBIO.COM)

Hardware

Interfaccia tra i livelli Interrupt Software
sia DOS, sia BIOS

Uso di **TRAP** invocabili da Assembler
ma anche da linguaggi di alto livello, p.e. C

Software di Sistema (DOS)



Funzioni DOS:
- fanno parte del sistema operativo
- risiedono nella RAM
- interfaccia ad alto livello
- dipendenza parziale dall'hardware
- manuale "DOS Technical Reference"

Funzioni BIOS:
- **B**asic **I**nput/**O**utput **S**ystem
- risiedono nella ROM
- interfaccia a basso livello
- dipendenza totale dall'hardware
- manuale "BIOS Interface Technical Reference"

Chiamata alle funzioni di sistema

Il trasferimento del controllo avviene mediante interrupt software
- separazione netta degli ambienti
- possibilità di variare dinamicamente la routine di servizio

interrupt inferiori **(<8)** situazioni anomale
interrupt hardware da **8** a **0F**h (eventi asincroni)

Funzioni DOS:
- interrupt da 20h a 3Fh
- interrupt 21h (DOS Function Call)
- funzionalità:
  - gestione video (caratteri) e tastiera
  - gestione file (indirizzi logici)
  - gestione memoria
  - funzioni che operano su date e tempi
  - trasferimento controllo ad altri programmi
  - terminazione programma

Funzioni BIOS (interrupt < 20H):
- interrupt 5h e da 10h a 1Ah
- funzionalità:
  - gestione video (caratteri e pixel) e tastiera
  - gestione dischi, porte seriali, stampanti, ...
  - gestione memoria
  - gestione tempo

DOS (Disk O. S.)

DOS funzionalità logiche di sistema
- gestori degli errori;
- primitive sui file (open, close)
- operazioni sui dispositivi
- operazioni sul sistema:

tempo, data, interrupt vector, memoria, direttori;

Interfaccia : TRAP
interrupt software dal **20**H al **27**H
interrupt software **21**H, function call

Struttura a livelli: **BIOS**, **DOS**
La driver keyboard BIOS ha un **buffer** per la tastiera e riconosce alcuni caratteri:

<ctrl><alt><del> *riinizializzazione di sistema: invoca il gestore del reset*
<alt><cifre tastiera numerica> *ricezione di un carattere decimale*
<printscreen> *stampa del video su stampante*
<ctrl> <break> *azione di abort del comando*

La routine BIOS attende l'arrivo del carattere
DOS fornisce un più alto livello di astrazione
DOS ha maggiore astrazione rispetto al BIOS
Le funzioni DOS sono più coordinate e su più dispositivi caso della **keyboard**

Creazione di astrazioni

la keyboard è coordinata con il video a produrre la unità astratta console
input di caratteri con o senza eco, input con/senza verifica del <ctrl> <break>, reset del buffer, etc.

INT 21H; function call del DOS
selezionate in base al valore contenuto in **AH**

```
; keybinput   AH <- 1
; char = Keybinput (); il carattere in AL
; input e echo del carattere  e check <ctrl><break>
MOV AH, 1
INT    21H

; keyboutput   AH <- 2
; char = Keybioutput ();
; output del carattere in DL
MOV AH, 2
MOV DL, 'a'
INT    21H

; buffinput  AH <- 0Ah
; punt  = Buffinput ();  in DS:DX
; input di caratteri, specificati in punt
; punt --> #daleggere, #letti, caratteri
MOV AH, 1
MOV DX, offset punt
INT 21H
```

DOS Function Call - INT 21h

```
mov   AH, <numero della funzione>
int   21h
```

1h Input da tastiera di un carattere con eco
8h Input da tastiera di un carattere senza eco
AL <- carattere letto

```
mov   AH, 1
int   21h
mov   byte ptr [DI], AL
```

2h Output di un carattere su video
5h Output di un carattere su stampante
DL deve contenere il carattere

```
mov   AH, 2
mov   DL, '$'
int   21h
```


9h Output di una stringa di caratteri che ha come terminatore un carattere '$'
DS:DX deve puntare alla stringa

```
str   db    "Stringa da stampare",'$'
pstr  dd    str
      ...
      mov   AH, 9
      lds   DX, pstr
      int   21h
```

4Ch Exit - il controllo ritorna a chi ha attivato il programma (di norma il DOS)
AL contiene il return code

```
mov   AH, 4Ch
mov   AL, returnCode
int   21h
```

Esempi di servizi BIOS

int **10**H; azioni su video
int **13**H; azioni su disco
int **14**H; azioni su porta seriale
int **16**H; azioni su tastiera
int **17**H; azioni su stampante
Passaggio parametri attraverso registri;

Acquisizione dello stato dei tasti speciali
; invia AH <- 2
; statoAL = invia (carattere)

```
        mov    AH, 2
        int    16h
```

in uscita, AL contiene lo stato dei tasti speciali:
- bit 0 = 1 se tasto Shift destro premuto
- bit 1 = 1 se tasto Shift sinistro premuto
- bit 2 = 1 se tasto Ctrl premuto
- bit 3 = 1 se tasto Alt premuto
- bit 4 = 1 se Scroll Lock attivo
- bit 5 = 1 se Num Lock attivo
- bit 6 = 1 se Caps Lock attivo
- bit 7 = 1 se stato Insert attivo

Invia un carattere alla stampante
; invia AH <- 0
; stato = invia (carattere)

```
        mov    AH, 0
        mov    AL, byte
        int    17h
```

in uscita, AH contiene il risultato dell'operazione:
- bit 0 = 1 se tempo scaduto (time out)
- bit 5 = 1 se fine carta
- bit 6 = 1 se acknowledgement (successo)

Scrittura di un carattere su video
; invia AH <- 9
; invia (carattere, #pag, attributo, #caratteri)

```
;          AL              BH        BL         CX
MOV  AH, 9
MOV  AL, char
MOV  BX, 0
MOV  CX, cont
INT    10H; BIOS video
```

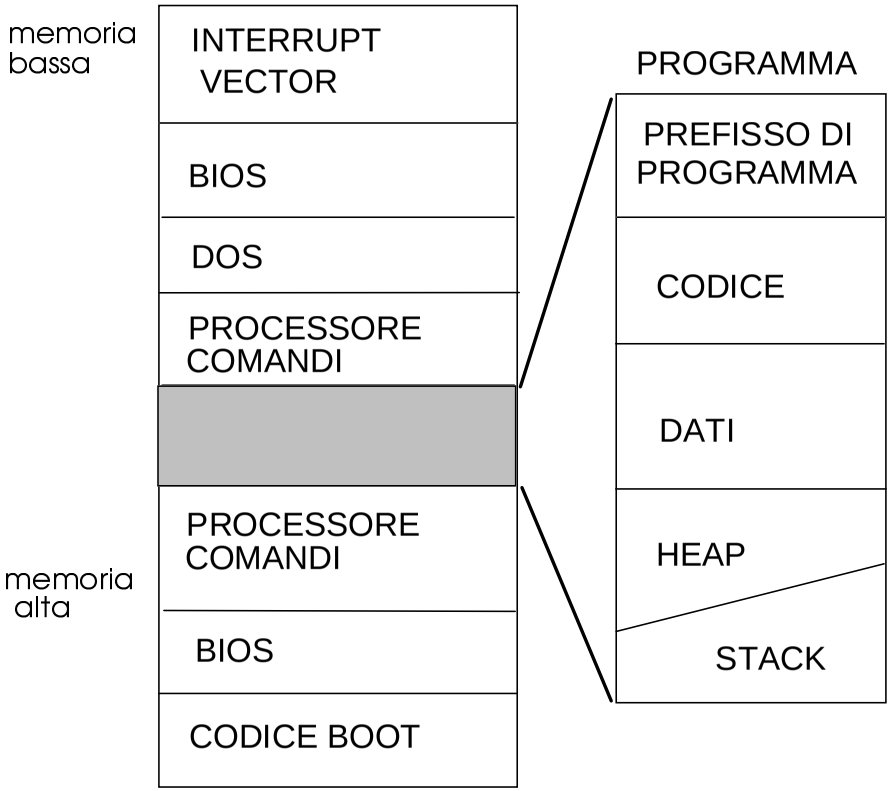
BIOS in ROM

Il file IBMBIO.COM consente la configurazione del sistema per ogni macchina
Per esempio il buffer di console viene creato secondo le direttive di questo file

Lettura di un carattere tastiera 16H
; ricevi AH <- 0
; codicetasto, carattere = ricevi ()
; AH AL
MOV AH, 0
INT 16H; BIOS tastiera
; attesa del carattere
; in AL codice carattere ASCII

All'interno il BIOS crea un buffer destinato a contenere le parole composte di caratteri e tasto letti. Sono poi forniti secondo necessità alle richieste. Ogni richiesta a buffer vuoto viene posta in attesa.
IBMBIO.COM specifica l'ampiezza del buffer e il gestore ad interruzione della tastiera
È possibile anche aggiungere nuove driver per i dispositivi

INIZIALIZZAZIONE



Program Segment Prefix (PSP)
-> gestori terminazione e aborto
-> un puntatore all'environment del programma

Alla INIZIALIZZAZIONE

- Si carica il **bootstrap**

- Questo carica il **BIOS** (IBMBIO.COM), le aree dinamiche del BIOS ed i suoi interrupt vector (il codice del BIOS è in memoria ROM)
- Si carica il **DOS** (IBMDOS.COM): il codice, le aree dati e gli interrupt vector
- Si carica il **processore comandi** (COMMAND.COM) in varie zone della memoria
  Parte nella memoria bassa, parte a partire dall'alto

Alcuni comandi sono parte del processore (**interni**), altri devono essere caricati su richiesta (**esterni**)

- Si caricano i **programmi utente** o
  i **comandi esterni**

Si carica allocando tutta
la memoria disponibile
al programma corrente

per ogni attivazione si genera sempre uno spazio **PSP** (**P**rogram **S**egment **P**refix) di comunicazione con il **DOS**

DUE tipi di FILE ESEGUIBILI

- file con estensione COM
descrivono un solo segmento e rilocabili all'atto del caricamento

una vera e propria immagine della memoria che il caricatore deve allocare

La terminazione del programma si ha con l'invocazione della exit (terminate)
INT 21h, AH <- 0

- file con estensione EXE
si descrivono più segmenti che vengono definiti ed allocati dal caricatore
I segmenti possono essere allocati dovunque in memoria o in modo assoluto

L'immagine del programma è sempre la stessa

Prefisso Programma più codice e dati
(eventualmente segmentato)

Direttiva MACRO

```
MUL4  MACRO
      xor     dx, dx
      shl     ax, 1
      rcl     dx, 1
      shl     ax, 1
      rcl     dx, 1
      ENDM
```

```
Mul4  PROC
      xor     dx, dx
      shl     ax, 1
      rcl     dx, 1
      shl     ax, 1
      rcl     dx, 1
      ret
Mul4  ENDP
```

```
      . . .

      mov     ax, Var
      MUL4                        call Mul4
      mov     word ptr [Result], ax
      mov     word ptr [Result+2], dx
```

Procedura:
- assemblata 1 sola volta - **più compatta**
- **più rigida**

Macro:
- assemblata + volte (ma senza CALL e RET)
- **più veloce**
- **più flessibile**


```
clear   MACRO  reg
        xor    reg, reg
        ENDM

mul2    MACRO
        shl    ax, 1
        rcl    dx, 1
        ENDM

mul4    MACRO
        clear  dx
        mul2
        mul2
        ENDM


push_rx  MACRO
        push   ax
        push   bx
        push   cx
        push   dx
        ENDM

pop_rx  MACRO
        pop    dx
        pop    cx
        pop    bx
        pop    ax
        ENDM
```

Direttiva REPT

Ripete un blocco di istruzioni

```
        REPT  espressione
           istruzioni
        ENDM
```

```
IPar    =       0
        REPT  10
        DW    IPar
IPar    =       IPar + 1
        ENDM
```

equivale a:

```
        DW    0
        DW    1
        . . .
        DW    9
```

```
mul2n MACRO       n
        clear   dx
        REPT  n
        mul2
        ENDM
        ENDM
```


Direttiva IRP (e IRPC)

Ripete un blocco di istruzioni con sostituzione

```
        IRP  parametro,<arg1[,arg2...]>
           istruzioni
        ENDM
```

```
        IRP     p,<0,5,33,2,88>
        DW      p
        ENDM
```

equivale a:

```
        DW      0
        DW      5
        DW      33
        DW      2
        DW      88
```

```
push_rx  MACRO
        IRP     reg,<ax,bx,cx,dx>
        push    reg
        ENDM
        ENDM
```